

3-3 · DuckDB를 이용한 로컬 대용량 데이터 처리 · 이론 1H

# DuckDB 개요와 열 지향 저장

서버 없이 100만 행을 — 열 지향이 빠른 이유

수도권ICT 이노베이션스퀘어  
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

# 1시간 뒤에는

---

- ① DuckDB 가 무엇이고 Pandas 와 역할이 어떻게 다른지 말할 수 있다
- ② 행 지향과 열 지향 저장의 차이를 서랍 비유로 설명한다
- ③ 파일 경로를 FROM 에 바로 적는 문법을 안다
- ④ zstd 를 기본 압축으로 고르는 이유를 안다

다음 1H 실습에서 100만 행 CSV 를 적재 절차 없이 바로 질의합니다

## 1. 서버가 없는 DB

# 질의 한 번까지 — MySQL 다섯 단계, DuckDB 두 단계

## 2주차 MySQL

컨테이너·포트·계정·테이블·INSERT 를 거쳐야 질의



VS

## DuckDB

그 절차가 통째로 없다 — 파일이 곧 테이블



# MySQL · PostgreSQL 과 DuckDB — 여섯 가지 차이

|        | MySQL · PostgreSQL | DuckDB             |
|--------|--------------------|--------------------|
| 형태     | 서버 프로세스            | 내 프로그램 안의 라이브러리    |
| 설치     | 컨테이너·설치 프로그램       | pip install duckdb |
| 접속     | 호스트·포트·계정          | 없음 — 그냥 함수 호출      |
| 데이터 넣기 | INSERT 해야 질의 가능    | 파일을 FROM 에 바로 적는다  |
| 잘하는 일  | 많은 사람이 조금씩 읽고 쓰기   | 한 사람이 많이 읽고 집계하기   |
| 저장 방식  | 행 지향               | 열 지향               |

OLTP(주문 하나 넣고 빼기 · MySQL) 와 OLAP(수백만 건 훑어 집계 · DuckDB) — 같은 SQL, 목적과 저장 방식이 다르다

# 서랍 정리 비유 — 사람별 서랍 vs 품목별 서랍

## 행 지향 — 사람별 서랍

한 사람의 소지품을 통째로 한 서랍에

- 「3번 손님의 모든 정보」를 꺼낼 때 빠르다
- 주문 하나 넣고 빼는 업무 처리에 맞다
- 일반 업무용 DB(PostgreSQL 등)

VS

## 열 지향 — 품목별 서랍

양말 서랍·티셔츠 서랍처럼 같은 종류끼리

- 「모든 손님의 양말만」 꺼낼 때 다른 서랍은 안 연다
- 분석 질의 = 특정 열 전체의 합계·평균
- DuckDB · Parquet 가 이 방식



## 열 지향이 분석에 유리한 두 가지 효과 — 작아지고, 빨라진다

- 1 필요한 열만 읽는다**  
합계·평균은 한 열만 보면 되니 나머지 서랍은 열지 않는다
- 2 같은 종류끼리 모이면 압축이 잘 된다**  
비슷한 값이 연달아 있어 파일이 작아진다
- 3 작아지니 읽을 양이 줄어 더 빨라진다**  
두 효과를 오늘 실습에서 각각 수치로 쟀다

오늘 쟀 것

크기 배율  
속도 배율

# 이런 일에는 어느 도구 — 분업 기준 여섯 줄

| 이런 일에는         | 도구     | 왜                         |
|----------------|--------|---------------------------|
| 수백만 행 필터·집계    | DuckDB | 필요한 열만 읽고, 메모리보다 커도 된다    |
| 여러 파일을 한 번에 질의 | DuckDB | FROM 'data/*.parquet' 한 줄 |
| SQL 로 표현되는 변환  | DuckDB | 읽기 쉽고 빠르다                 |
| 행마다 다른 규칙 적용   | Pandas | apply·정규식·조건 분기가 자유롭다     |
| 외부 라이브러리와 연동   | Pandas | 시각화·머신러닝이 DataFrame 을 받는다 |
| 결측·이상치 세밀 처리   | Pandas | 함수가 풍부하다                  |

「둘 중 뭐가 좋은가」가 아니라 분업 — 100만 행을 DuckDB 로 1만 행까지 줄이면 Pandas 가 편하게 다룬다(3-5 에서 만든다)

#### 4. 파일을 테이블처럼

# 파일 경로를 FROM 에 그대로 적는 문법 네 가지

**1 파일 하나 — FROM 'events.csv'**  
SELECT count(\*) FROM 'events.csv'; — CSV 든 Parquet 든 경로가 곧 테이블

**2 여러 파일 — FROM 'data/\*.parquet'**  
와일드카드 한 줄로 폴더의 파일을 한꺼번에 질의

**3 옵션 주기 — read\_csv('events.csv', sample\_size = -1)**  
함수 형태로 적으면 타입 추론 표본 같은 옵션을 줄 수 있다

**4 내보내기 — COPY (SELECT ...) TO 'out.parquet'**  
(FORMAT parquet) — 질의 결과를 파일로 바로 저장

경로는 '지금 프로그램이 실행되는 폴더' 기준 — 다른 폴더에서 돌리면 파일을 못 찾는다. 오늘 가장 많이 만날 오류, 헛갈리면 절대 경로로



5. 압축 방식

# zstd · snappy · gzip · 없음 — 이번 과정의 기본은 zstd

| 압축     | 크기    | 속도     | 언제                  |
|--------|-------|--------|---------------------|
| zstd   | 가장 작다 | 보통     | 저장 비용이 중요할 때(기본 권장) |
| snappy | 중간    | 가장 빠르다 | 자주 읽는 중간 산출물        |
| gzip   | 작다    | 느리다    | 호환성이 필요할 때          |
| 없음     | 가장 크다 | 가장 빠르다 | 거의 안 쓴다             |

4주차 Iceberg 도 같은 선택 — 심화 과제에서 snappy 와 직접 비교해 볼 수 있다

# 이 1시간에서 꼭 가져갈 것

---

- 1 DuckDB 는 서버 없이 파이썬 안에서 도는 분석용 DB 다
- 2 분석 질의는 「특정 열 전체」를 보므로 열 지향이 유리하다
- 3 열 지향은 작아지고(압축) 빨라진다(읽을 양 감소)
- 4 파일 경로를 FROM 에 바로 적고, 경로는 실행 위치 기준이다
- 5 대용량은 DuckDB 로 줄이고, 세밀한 가공은 Pandas 로 한다

### 3-3 · 이론 1H 마무리

## 다음은 파일 직접 질의 실습 (1H)

100만 행 events.csv 를 만들고, 적재 절차 없이 SQL 로 바로 질의해 봅니다

✓ pip install duckdb 와 버전 확인

✓ 시드 고정 100만 행 CSV 생성

✓ DESCRIBE·GROUP BY·CTE·Window 그대로

✓ .df() 로 Pandas 왕복

준비할 것

venv 활성화 · 디스크 여유 1GB 이상